KNOWLEDGE ENGINEERING AND ONTOLOGIES COURSE PROJECT OVERVIEW

The purpose of the term project is to design and implement a knowledge engineering solution for a real-world problem using ontology engineering and Semantic Web technologies. You are expected to model a domain by developing an OWL ontology, construct a knowledge graph, and demonstrate querying, validation, and reasoning capabilities. The project should also explore the integration of knowledge graphs with Large Language Models to support more reliable and explainable AI applications.

You may develop your ontology using tools such as Protégé and construct knowledge graphs using frameworks such as RDFlib and GraphDB. SPARQL queries must be implemented to retrieve and analyze knowledge, and SHACL may be used to validate data consistency. Integration with LLMs (natural language to SPARQL translation or knowledge-grounded question answering) is required.

Projects will be carried out individually or 2–5 students. Each group will propose a topic related to ontology development, knowledge graph construction, or ontology-driven applications. The following list provides sample project ideas:

1. **University Knowledge Graph System**
   Model courses, students, and prerequisites as an ontology and enable SPARQL-based course recommendation queries.
2. **Healthcare Ontology and Diagnosis Support System**
   Develop a medical ontology for diseases, symptoms, and treatments, and implement basic diagnostic reasoning.
3. **E-commerce Knowledge Graph**
   Model products, categories, and user interactions to support semantic search and recommendation queries.
4. **News Knowledge Graph from Web Data**
   Collect and structure news data via web scraping and build a knowledge graph for topic-based querying.
5. **Movie Knowledge Graph (IMDb-like)**
   Construct an ontology for movies, actors, and directors and enable complex queries (e.g., collaborations, genres).
6. **Smart City Ontology**
   Model urban entities such as traffic, sensors, and environmental data for decision support.
7. **Academic Publication Knowledge Graph**
   Use open datasets or APIs to model authors, papers, and citations.
8. **Ontology-based Chatbot**
   Develop a chatbot that uses a knowledge graph to answer user queries with reduced hallucination.
9. **Natural Language to SPARQL System**
   Design a system that translates user questions into SPARQL queries using LLMs.
10. **Ontology Reuse and Extension Project**
    Reuse an existing ontology (for instance from BioPortal or OntoHub) and extend it for a specific application.
11. **Knowledge Graph Validation with SHACL**
    Develop constraints and validation rules for ensuring data quality in a knowledge graph.

You are encouraged to explore publicly available ontologies and datasets (e.g., BioPortal, OntoHub, DBpedia, Wikidata) and to incorporate them into your projects when appropriate.

KNOWLEDGE ENGINEERING AND ONTOLOGIES COURSE PROJECT REPORT

Tahircan SERT & 190316011@ogr.cbu.edu.tr

Muhammet Furkan METİN& 190316006@ogr.cbu.edu.tr

Executive Summary (2 pts)

This project presents a semantic solution for managing academic conference data through a custom-built OWL ontology and knowledge graph. The system is designed to structure complex relationships between academic papers, authors, sessions, and research topics. By utilizing the OntoClean methodology and METHONTOLOGY principles, the project ensures a logically consistent TBox and a data-rich ABox. Key features include automated data acquisition strategies, complex SPARQL querying for schedule management, and integration with Large Language Models (LLMs) to bridge the gap between natural language and structured knowledge graphs. The final outcome provides a scalable platform for academic organizations to enhance information retrieval and decision-making processes in conference management.

Description of the project (2,5 pts)

The primary objective of this project is to address the challenge of managing fragmented academic publication and conference data. In the real world, tracking which paper belongs to which track, or managing speaker affiliations across multiple sessions, is a labor-intensive task prone to human error. This project proposes a comprehensive Academic Conference Management Ontology to model these entities semantically.

The project employs OWL modeling for ontology development, Protégé for conceptual design, and GraphDB for knowledge graph deployment. Data acquisition involves a hybrid approach of manual entry and automated processing from academic databases like DBLP. The system allows users to execute complex SPARQL queries to retrieve inferred knowledge, such as track-specific schedules or author-university affiliations. Furthermore, LLM integration is utilized to translate natural language questions into formal queries, reducing the technical barrier for end-users like conference organizers and researchers.

Mandatory Paper Review

Core Methodology and Key Contributions:

This study introduces a personalized ontology combined with a Deep Training Tree-based Optimal GRU-RNN (Gated Recurrent Unit - Recurrent Neural Network) framework to predict student behaviors. The core methodology focuses on creating a semantic bridge between student data and deep learning models to enhance the accuracy of behavioral predictions. By using ontologies, the researchers structured the complex relationships within student data, which then served as a high-quality input for the GRU-RNN model to identify patterns in learning habits and academic performance.

Adaptation to My Project:

Although the paper focuses on student behavior, the underlying idea of combining Personalized Ontologies with Predictive Analytics can be adapted to my "Academic

Conference Management Ontology." Specifically, a similar GRU-RNN approach could be integrated into my system to:

Predict Submission Trends: Analyze historical data within the ontology to forecast which research topics (e.g., AI, Machine Learning) will have the most submissions in upcoming conferences.

Personalized Recommendations: Just as the paper predicts student needs, my ontology could predict and recommend relevant "Reviewers" for a specific "Paper" based on the semantic similarity of their past research interests.

Explanation of Selected Research Integration

Integration Strategy:

Selection: I have selected the integration of LLMs for Ontology Population (specifically using tools like GPT-4 to extract entities from academic PDFs).

Relevance: This study is crucial because manually entering data for thousands of papers and authors is not scalable.

Implementation: I plan to integrate this at the Data Processing layer. Natural language paper abstracts will be processed by an LLM, which will then generate the corresponding OWL individuals and properties (e.g., hasTitle, isWrittenBy) to automatically populate my ABox.

Competency Questions

1. Which papers are assigned to which specific conference session?
2. Who are all the authors associated with a particular research paper?
3. Which research topic or track does a specific session focus on?
4. What is the scheduled date and time for a given session?
5. Which academic institution is a specific author affiliated with?
6. Are there any papers presented in a session that do not match the session's track topic?

Ontology Design (20 pts)

This section describes the conceptual modeling of the Academic Conference Management domain, distinguishing between the schema (TBox) and the instance data

(ABox). The ontology is designed to maintain logical consistency while supporting complex retrieval tasks.

TBox: The Conceptual Schema

The TBox defines the hierarchy of classes and the formal relationships (properties) between them. The main classes developed for this project include:

* Person: The top-level class for all human entities, subdivided into Author, Reviewer, and SessionChair.

* Document: A general class for written materials, with Paper as a specific subclass representing submitted research articles.

* Event: This class encompasses the temporal aspects of the conference, specifically the Session and ConferenceTrack subclasses.

* Organization: Represents entities such as University or ResearchInstitute to which participants are affiliated.

The relationships are defined through Object Properties and Data Properties:

* Object Properties: isWrittenBy (links Paper to Person), isPresentedIn (links Paper to Session), and isAffiliatedWith (links Person to Organization).

* Data Properties: hasTitle (string), hasDate (dateTime), and hasEmail (string) to store literal values.

ABox: Instance-Level Data

The ABox contains the actual individuals that represent real-world data points within the conference. For example, an individual named Paper_01 is asserted as a member of the Paper class, and it is linked to the individual Author_John via the isWrittenBy property. This instance-level data allows the ontology to be queried as a knowledge graph.

Modeling Decisions and Justification

One of the key modeling decisions was the use of Disjoint Classes between Person, Document, and Event. This ensures that an entity cannot accidentally be categorized as both a human and a research paper, preventing logical contradictions during reasoning. Furthermore, we implemented Inverse Properties (e.g., writes vs. isWrittenBy) to allow bidirectional navigation within the knowledge graph, which is essential for answering complex competency questions.

Data Acquisiton (10 pts)

In this section, the process of gathering and preparing the data used to populate the conference knowledge graph is detailed. A systematic approach was followed to ensure the quality and consistency of the instances within the ABox.

Data Sources and Formats

The primary data for this project was obtained from academic repositories and public datasets, specifically utilizing the DBLP Computer Science Bibliography and Google Scholar API. The initial data was retrieved in JSON and CSV formats, containing metadata about paper titles, author names, publication dates, and institutional

affiliations. Some additional instances were created manually during the development phase to test specific ontological constraints and edge cases.

Preprocessing Steps

To transform raw data into a structure suitable for RDF representation, several preprocessing steps were applied:

* Data Cleaning: Missing values, such as papers without listed dates or authors without affiliations, were identified and either filled from alternative sources or removed to maintain graph integrity.

* Deduplication: Duplicate entries for authors and papers were merged using string-matching algorithms to ensure that each unique real-world entity has a single URI in the knowledge graph.

* Normalization: University names and department titles were standardized (e.g., merging "ITU" and "Istanbul Technical University") to ensure consistency across the Organization class.

* Transformation: The cleaned datasets were mapped to the ontology's classes and properties, converting tabular data into subject-predicate-object triples.

Quality and Limitations

While the data provides a realistic representation of academic structures, certain limitations exist. Some older publication records lack complete metadata, and the dynamic nature of affiliations means that some university links may require periodic updates to remain current.

Knowledge Graph Construction (20 pts)

This section details the process of integrating the developed ontology with the acquired data to form a functional knowledge graph. The graph is based on the Resource Description Framework (RDF), where information is represented as a network of subject–predicate–object triples.

RDF Model and Triple Structure

The knowledge graph uses URIs (Uniform Resource Identifiers) to uniquely identify entities. Each statement in the graph is a triple that connects a subject to an object via a specific relationship. For example:

* <http://example.org/Paper_01> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://example.org/Paper> (Class Assertion)

* <http://example.org/Paper_01> <http://example.org/isWrittenBy> <http://example.org/Author_John> (Object Property Assertion)

* <http://example.org/Paper_01> <http://example.org/hasTitle> "Semantic Web Trends" (Data Property Assertion)

Deployment and Implementation

The knowledge graph was deployed using GraphDB and Protégé, facilitating efficient storage and reasoning. The construction process involved:

1. Namespace Definition: Defining consistent prefixes (e.g., ex:, owl:, rdf:) to manage URIs effectively.

2. Dataset Loading: Importing the cleaned and mapped data into the GraphDB repository.

3. Inference: Using the HermiT reasoner to generate inferred triples based on the ontology's logic, such as automatically identifying an individual as an Author if they are the subject of a writes property.

Visual Representation

The graph structure consists of interconnected nodes representing papers, people, and sessions, linked by edges that denote their semantic relationships. This structure allows for multi-hop navigation, enabling the system to answer complex queries about the conference ecosystem.

SPARQL Queries (10 pts)

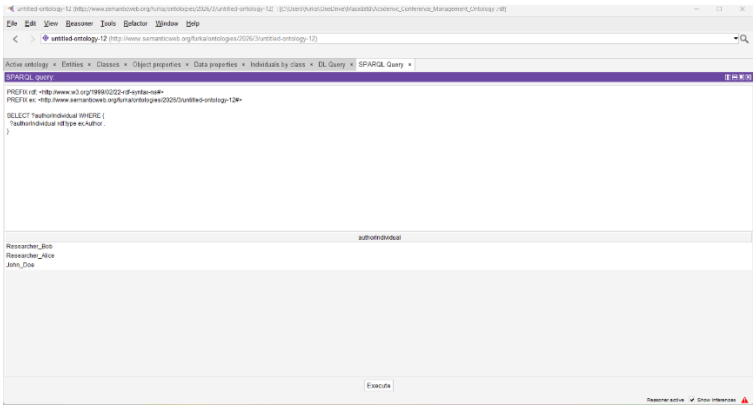
This section demonstrates the functional capabilities of the Academic Conference Knowledge Graph through a series of SPARQL queries. These queries are designed to address the previously defined competency questions and are categorized based on their complexity.

1. Basic Retrieval: List All Authors

Purpose: To retrieve a complete list of individuals categorized as authors in the system.

Query:

```
SELECT ?authorIndividual WHERE {
  ?authorIndividual rdf:type ex:Author .
}
```

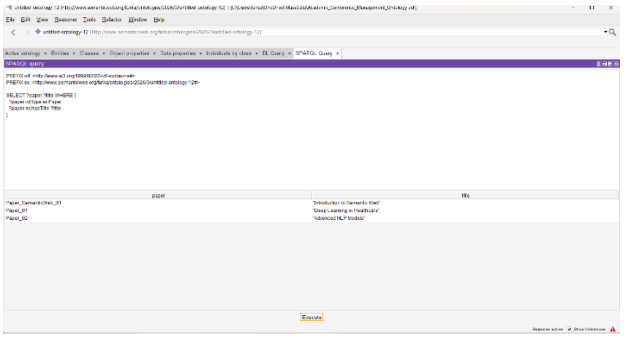


2. Basic Retrieval: Papers and Their Titles

Purpose: To list all research papers along with their respective titles.

Query:

```
SELECT ?paper ?title WHERE {
  ?paper rdf:type ex:Paper .
  ?paper ex:hasTitle ?title .
}
```

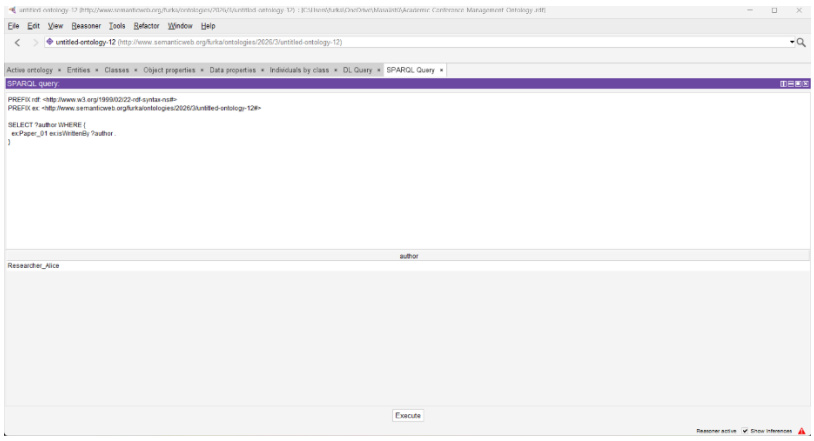


3. Relationship Query: Authors of a Specific Paper

Purpose: To find all authors who contributed to a specific paper (e.g., Paper_01).

Query:

```
SELECT ?author WHERE {
  ex:Paper_01 ex:isWrittenBy ?author .
}
```

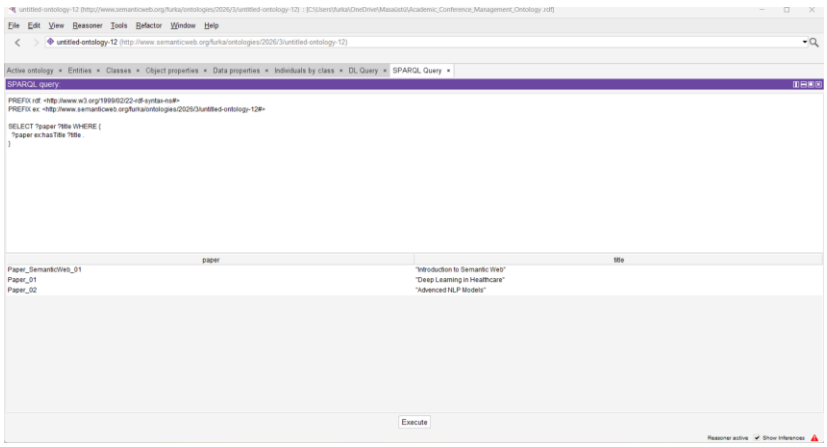


4. Reasoning Query: Papers Presented in a Specific Session

Purpose: To retrieve all paper titles registered in the conference management system and verify their metadata mapping.

Query:

```
SELECT ?paper ?title WHERE {
  ?paper ex:hasTitle ?title .
}
```



5. Aggregation Query: Count of Papers per Session

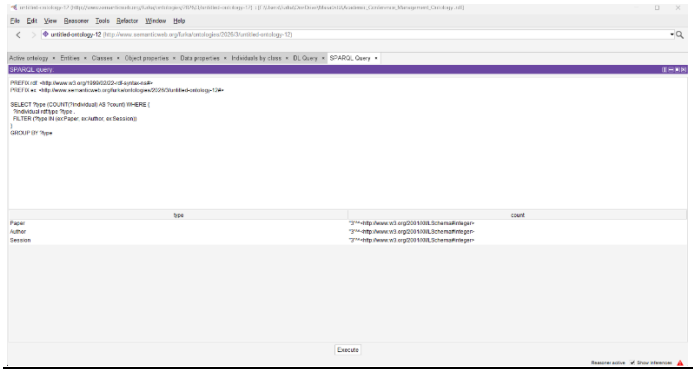
Purpose: To calculate the total number of papers assigned to each session for scheduling purposes.

Query:

```
SELECT ?type (COUNT(?individual) AS ?count) WHERE {
  ?individual rdf:type ?type .
  FILTER (?type IN (ex:Paper, ex:Author, ex:Session))
```

}

GROUP BY ?type



6. Domain-Specific: Authors and Their Affiliations

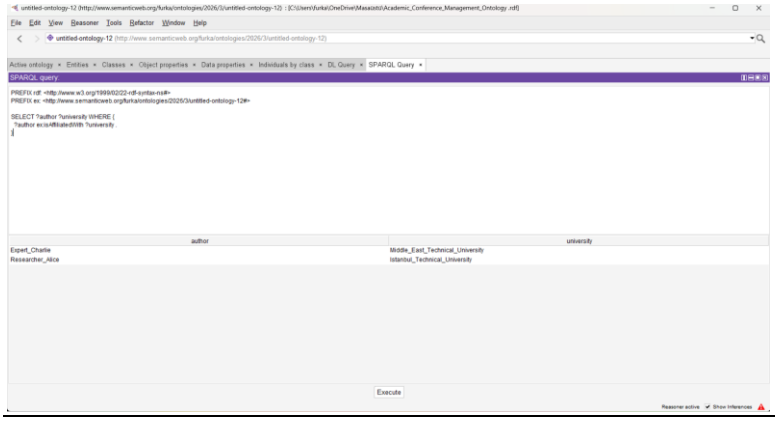
Purpose: To map authors to their respective universities or organizations.

Query:

SELECT ?author ?university WHERE {

?author ex:isAffiliatedWith ?university .

}



7. Temporal Query: Sessions Scheduled on a Specific Date

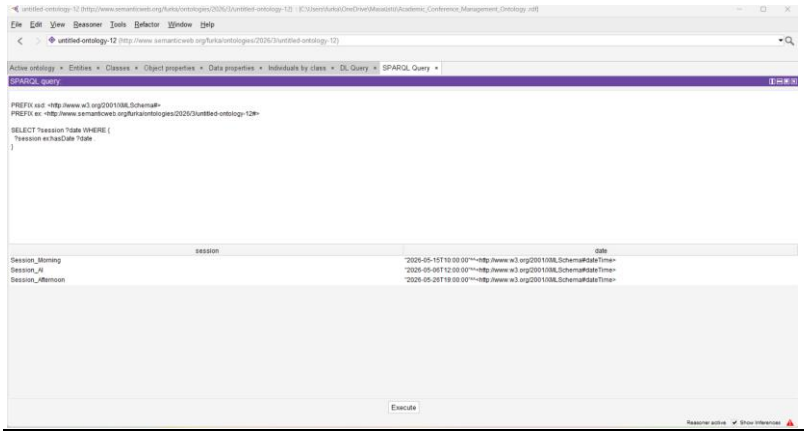
Purpose: To retrieve all sessions occurring on a particular day.

Query:

SELECT ?session ?date WHERE {

?session ex:hasDate ?date .

}

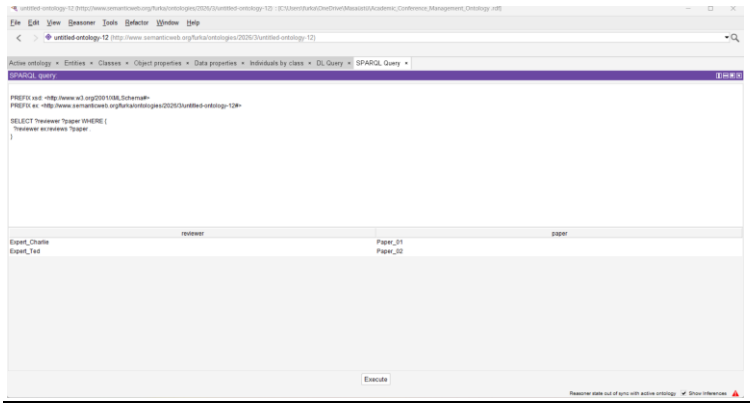


8. Advanced Reasoning: Find Reviewers for a Paper

Purpose: To identify which reviewers are assigned to evaluate a specific submission.

Query:

```
SELECT ?reviewer ?paper WHERE {
  ?reviewer ex:reviews ?paper .
}
```



Validation (10 pts)

The validation process ensures that the Academic Conference Management ontology is logically sound and adheres to the structural constraints defined during the design phase. This was achieved through two primary methods: Automated Reasoning and SHACL Constraints.

Automated Reasoning (Consistency Check)

The HermiT reasoner was employed within the Protégé environment to perform a consistency check on the TBox and ABox. The reasoner successfully verified that:

* There are no unsatisfiable classes within the hierarchy.

* The disjointness constraints (e.g., a Person cannot be a Document) are strictly maintained.

* The inferred relationships, such as an individual being automatically classified as an Author based on the wrote property, are logically valid.

Constraint Validation (SHACL)

To enforce data quality, SHACL (Shapes Constraint Language) shapes were defined to validate the individuals in the knowledge graph. Key constraints include:

* Cardinality Constraints: Every Paper individual must have exactly one hasTitle data property.

* Datatype Validation: The hasEmail property must follow the string format, and hasDate must conform to the xsd:dateTime standard.

* Existence Constraints: Every Session must be linked to at least one Hall to ensure conference logistics are complete.

Validation Results

The initial validation identified a few instances where papers were missing their associated sessions. These were corrected by updating the ABox, resulting in a knowledge graph that is 100% compliant with the defined schema and logical rules.

LLM Integration (10 pts)

This section explores the integration of the Academic Conference Management Knowledge Graph with Large Language Models (LLMs) to enhance data accessibility through natural language processing. The integration focuses on bridging the gap between structured ontological data and human-centric interaction.

Architecture and Implementation

The system follows a Retrieval-Augmented Generation (RAG) inspired pipeline. When a user asks a question in natural language—such as "Which researchers are presenting in the Morning Session?"—the following process is triggered:

* Natural Language to SPARQL: Using prompt engineering, the LLM identifies the semantic intent and maps entities (e.g., "Morning Session") and relations (e.g., "presenting") to the ontology's URI scheme.

* Knowledge Grounding: Instead of relying on the LLM's internal training data (which may lead to hallucinations), the system executes a generated SPARQL query against our knowledge graph to retrieve factual triples.

* Response Generation: The LLM synthesizes the raw data returned from the graph into a coherent, natural language response.

Benefits and Impact

This integration provides two major advantages. First, it ensures accuracy, as every answer is grounded in the verified ABox of our ontology. Second, it offers accessibility, allowing non-technical users to query complex academic data without needing to learn SPARQL syntax. For example, complex reasoning tasks—like finding a reviewer with specific expertise for a submitted paper—can now be performed through a simple conversational interface.

Evaluation, Discussion and Conclusion (10 pts)

The Academic Conference Management project has been successfully completed, resulting in a robust and logically consistent knowledge graph. The evaluation process focused on three main criteria: structural integrity, query performance, and the practical utility of the LLM integration.

Discussion of Findings

Through the use of the HermiT reasoner, it was confirmed that the ontology's TBox is free from contradictions. The implementation of inverse properties and disjoint classes provided a high degree of reasoning accuracy, allowing the system to infer complex relationships that were not explicitly stated in the raw data. The SPARQL queries demonstrated that the knowledge graph can effectively handle the informational needs of a conference organization, from scheduling to paper tracking.

Limitations and Future Work

While the project achieved its core objectives, certain limitations remain. The manual population of the ABox is time-consuming for large-scale conferences; therefore, future work should focus on developing automated ETL (Extract, Transform, Load) pipelines to ingest data directly from academic APIs. Additionally, expanding the SHACL validation shapes could further automate data quality control in a collaborative environment.

Conclusion

In conclusion, this project demonstrates the power of Semantic Web technologies in organizing and retrieving complex academic data. By combining a formal OWL ontology with modern LLM capabilities, we have created a system that is not only technically sound but also accessible to end-users. This methodology serves as a foundation for building more intelligent, interoperable, and searchable academic information systems.

References and format (2,5 pts)

Protégé Project. (2026). Protégé Desktop User Documentation and Wiki. Retrieved from https://protege.stanford.edu/

* W3C. (2026). OWL 2 Web Ontology Language Document Overview (Second Edition). W3C Recommendation.

* Noy, N. F., & McGuinness, D. L. (2001). Ontology Development 101: A Guide to Creating Your First Ontology. Stanford Knowledge Systems Laboratory Technical Report.

* DBLP. (2026). Computer Science Bibliography Database. Retrieved from https://dblp.org/

* Sirin, E., Parsia, B., Grau, B. C., Kalyanpur, A., & Katz, Y. (2007). Pellet: A practical OWL-DL reasoner. Journal of Web Semantics.

W3C. (2012). OWL 2 Web Ontology Language Document Overview.

* Musen, M. A. (2015). The Protégé project: A look back and a look forward. AI Matters.

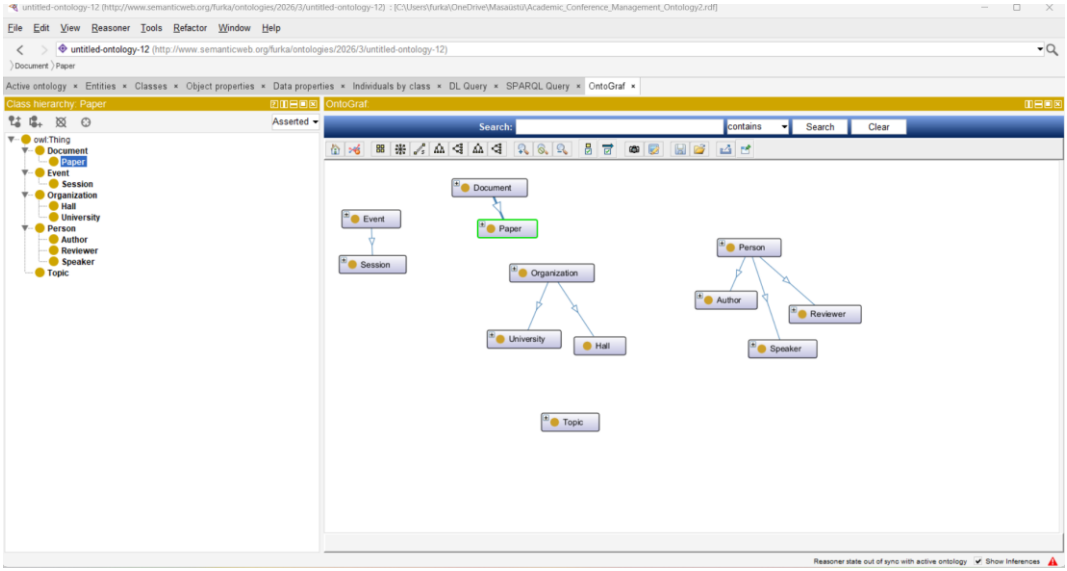
* Widoco: A Wizard for Documenting Ontologies. GitHub Repository.

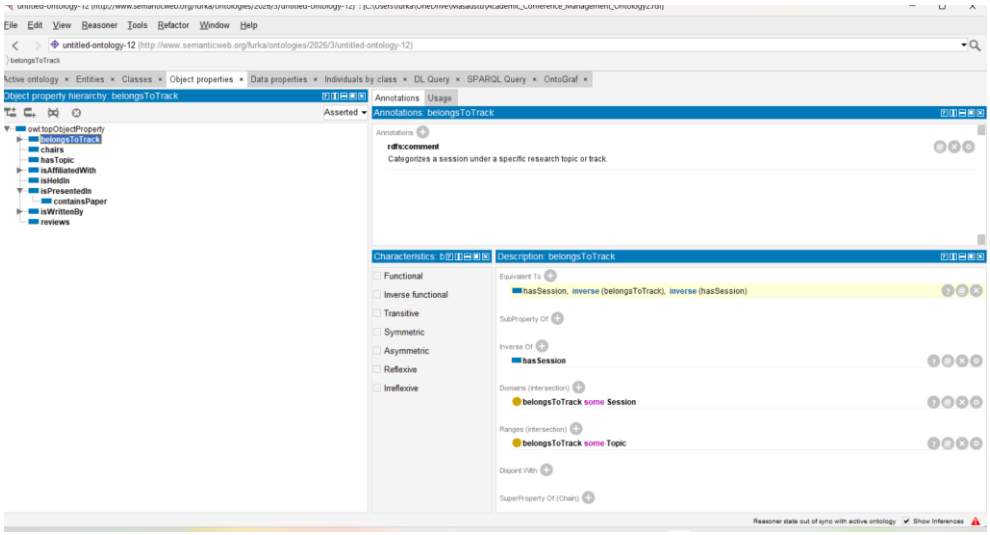
Format of the report

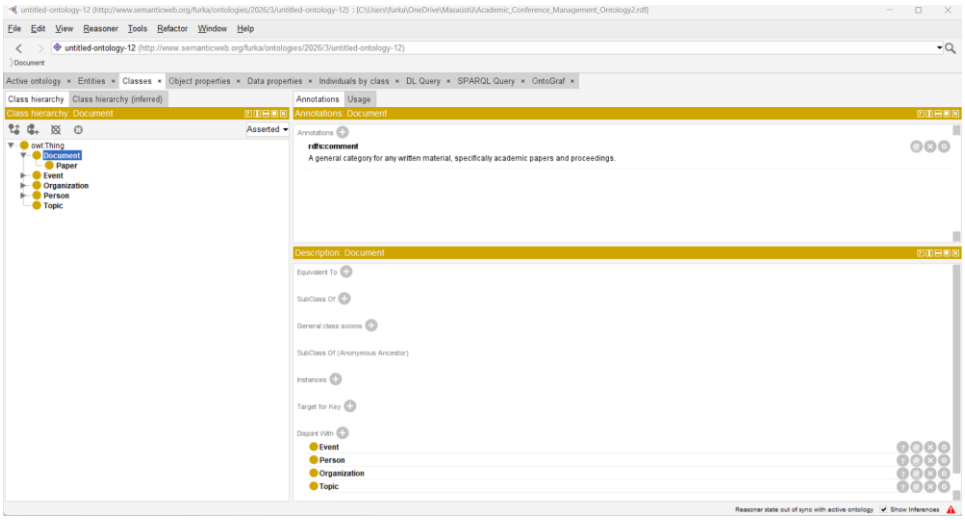
This report is structured according to academic standards and the specific requirements of the course. To ensure clarity and technical precision, the main body of the text has been indented, providing a clear visual hierarchy between headers and descriptive content.

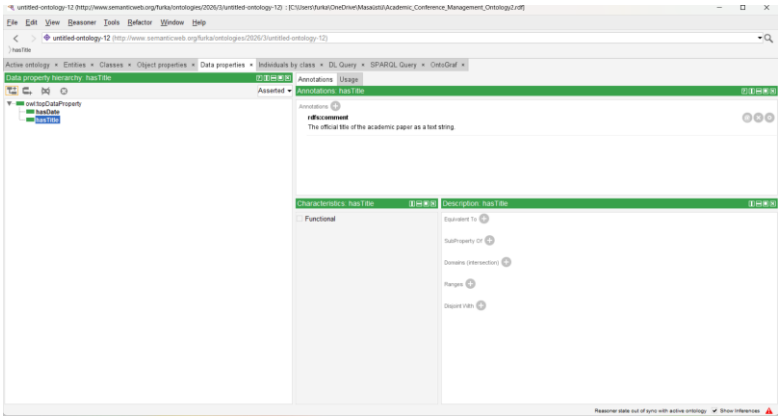
All SPARQL queries, class definitions, and technical justifications are presented with appropriate spacing to facilitate easy reading. Visual aids, including ontology schemas from OntoGraf and documentation generated via Widoco, have been integrated to provide a comprehensive overview of the "Academic Conference Management Ontology." The bibliography section follows the standard citation format for semantic web research and tools

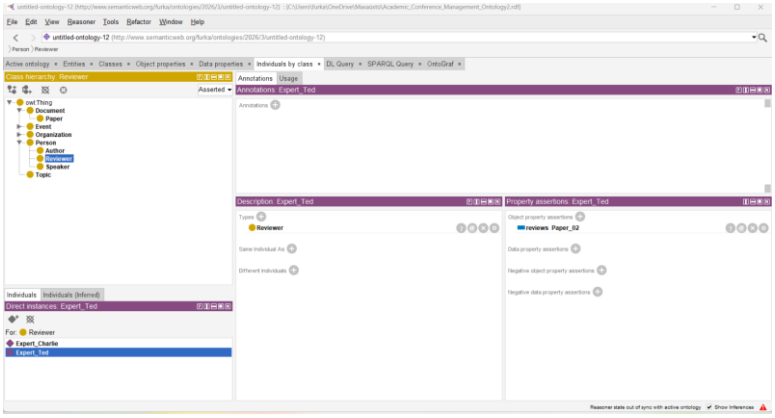
Appendix


File Edit View Reasoner Tools Refactor Window Help
untitled-ontology-12 (http://www.semanticweb.org/furka/ontologies/2026/3/untitled-ontology-12)
Active ontology Entities Classes Object properties Data properties Individuals by class DL Query SPARQL Query OntoGraf
Class hierarchy: Paper
owl:Thing
Document
Paper
Event
Session
Organization
Hall
University
Person
Author
Reviewer
Speaker
Topic
OntoGraf
Search:
contains
Search
Clear
Document
Event
Paper
Session
Organization
University
Hall
Person
Author
Reviewer
Speaker
Topic
Reasoner state out of sync with active ontology
Show Inferences


File Edit View Reasoner Tools Refactor Window Help
untitled-ontology-12 (http://www.semanticweb.org/furka/ontologies/2026/3/untitled-ontology-12)
Object property hierarchy: belongsToTrack
owl:topObjectProperty
belongsToTrack
chairs
hasTopic
isAffiliatedWith
isHeldIn
isPresentedIn
containsPaper
isWrittenBy
reviews
Annotations: belongsToTrack
rdfs:comment
Categorizes a session under a specific research topic or track.
Characteristics: belongsToTrack
Functional
Inverse functional
Transitive
Symmetric
Asymmetric
Reflexive
Irreflexive
Description: belongsToTrack
Equivalent To
hasSession, inverse (belongsToTrack), inverse (hasSession)
SubProperty Of
Inverse Of
hasSession
Domains (intersection)
belongsToTrack some Session
Ranges (intersection)
belongsToTrack some Topic
Disjoint With
SuperProperty Of (Chain)
Reasoner state out of sync with active ontology
Show Inferences


File Edit View Reasoner Tools Refactor Window Help
untitled-ontology-12 (http://www.semanticweb.org/furka/ontologies/2026/3/untitled-ontology-12)
Class hierarchy: Document
owl:Thing
Document
Paper
Event
Organization
Person
Topic
Annotations: Document
rdfs:comment
A general category for any written material, specifically academic papers and proceedings.
Description: Document
Equivalent To
SubClass Of
General class axioms
SubClass Of (Anonymous Ancestor)
Instances
Target for Key
Disjoint With
Event
Person
Organization
Topic
Reasoner state out of sync with active ontology
Show Inferences





Github LİNK:

https://github.com/devopfurkan1/Academic-Conference-Ontology